Volume 03 - Book 03.05 Capability System

Version v7 draft

README.md

Book 03.05 - Capability System

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-README

Purpose

Define the Capability System blueprint package and its subsystem decomposition as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for the Capability System blueprint package and its subsystem decomposition. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for the Capability System blueprint package and its subsystem decomposition.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for the Capability System blueprint package and its subsystem decomposition.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.

- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilitySystemDefined
- CapabilitySystemRegistered
- CapabilitySystemMatched
- CapabilitySystemInvoked
- CapabilitySystemRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability System has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

INDEX.md

Book 03.05 Index

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-INDEX

Purpose

Define the canonical reading order and document IDs for Capability System as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for the canonical reading order and document IDs for Capability System. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for the canonical reading order and document IDs for Capability System.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for the canonical reading order and document IDs for Capability System.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- Book0305IndexDefined
- Book0305IndexRegistered
- Book0305IndexMatched
- Book0305IndexInvoked
- Book0305IndexRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Book 03.05 Index has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010

- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-01-capability-system-overview.md

Book 03.05.01 - Capability System Overview

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-001

Purpose

Define the governed capability layer that packages reusable work outcomes across humans, agents, workflows, tools, knowledge, and models as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for the governed capability layer that packages reusable work outcomes across humans, agents, workflows, tools, knowledge, and models. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for the governed capability layer that packages reusable work outcomes across humans, agents, workflows, tools, knowledge, and models.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for the governed capability layer that packages reusable work outcomes across humans, agents, workflows, tools, knowledge, and models.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilitySystemOverviewDefined
- CapabilitySystemOverviewRegistered
- CapabilitySystemOverviewMatched
- CapabilitySystemOverviewInvoked
- CapabilitySystemOverviewRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability System Overview has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

Book 03.05.02 - Capability Graph

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-002

Purpose

Define the graph of capability relationships, prerequisites, substitutes, compositions, ownership, and maturity as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for the graph of capability relationships, prerequisites, substitutes, compositions, ownership, and maturity. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for the graph of capability relationships, prerequisites, substitutes, compositions, ownership, and maturity.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for the graph of capability relationships, prerequisites, substitutes, compositions, ownership, and maturity.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityGraphDefined
- CapabilityGraphRegistered
- CapabilityGraphMatched
- CapabilityGraphInvoked
- CapabilityGraphRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Graph has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-03-capability-registry.md

Book 03.05.03 - Capability Registry

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-003

Purpose

Define the canonical registry for capability definitions, versions, owners, contracts, and lifecycle state as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for the canonical registry for capability definitions, versions, owners, contracts, and lifecycle state. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for the canonical registry for capability definitions, versions, owners, contracts, and lifecycle state.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for the canonical registry for capability definitions, versions, owners, contracts, and lifecycle state.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityRegistryDefined
- CapabilityRegistryRegistered
- CapabilityRegistryMatched
- CapabilityRegistryInvoked
- CapabilityRegistryRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Registry has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-04-capability-discovery.md

Book 03.05.04 - Capability Discovery

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-004

Purpose

Define discovery of available capabilities by domain, task, role, agent, workflow, quality bar, and permission context as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for discovery of available capabilities by domain, task, role, agent, workflow, quality bar, and permission context. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for discovery of available capabilities by domain, task, role, agent, workflow, quality bar, and permission context.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for discovery of available capabilities by domain, task, role, agent, workflow, quality bar, and permission context.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityDiscoveryDefined
- CapabilityDiscoveryRegistered
- CapabilityDiscoveryMatched
- CapabilityDiscoveryInvoked
- CapabilityDiscoveryRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Discovery has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-05-capability-matching.md

Book 03.05.05 - Capability Matching

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-005

Purpose

Define matching work intent to suitable capabilities using requirements, constraints, context, policy, and evidence as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for matching work intent to suitable capabilities using requirements, constraints, context, policy, and evidence. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for matching work intent to suitable capabilities using requirements, constraints, context, policy, and evidence.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for matching work intent to suitable capabilities using requirements, constraints, context, policy, and evidence.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityMatchingDefined
- CapabilityMatchingRegistered
- CapabilityMatchingMatched
- CapabilityMatchingInvoked
- CapabilityMatchingRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Matching has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-06-capability-ranking.md

Book 03.05.06 - Capability Ranking

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-006

Purpose

Define ranking candidate capabilities by fit, risk, quality, cost, latency, availability, and governance readiness as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for ranking candidate capabilities by fit, risk, quality, cost, latency, availability, and governance readiness. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for ranking candidate capabilities by fit, risk, quality, cost, latency, availability, and governance readiness.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for ranking candidate capabilities by fit, risk, quality, cost, latency, availability, and governance readiness.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityRankingDefined
- CapabilityRankingRegistered
- CapabilityRankingMatched
- CapabilityRankingInvoked
- CapabilityRankingRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Ranking has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

Book 03.05.07 - Capability Requirements

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-007

Purpose

Define requirements that define capability inputs, outputs, constraints, permissions, evidence, and success measures as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for requirements that define capability inputs, outputs, constraints, permissions, evidence, and success measures. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for requirements that define capability inputs, outputs, constraints, permissions, evidence, and success measures.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for requirements that define capability inputs, outputs, constraints, permissions, evidence, and success measures.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityRequirementsDefined
- CapabilityRequirementsRegistered
- CapabilityRequirementsMatched
- CapabilityRequirementsInvoked
- CapabilityRequirementsRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Requirements has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

Book 03.05.08 - Capability Dependencies

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-008

Purpose

Define dependency tracking between capabilities, skills, models, knowledge, runtimes, workflows, and policies as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for dependency tracking between capabilities, skills, models, knowledge, runtimes, workflows, and policies. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for dependency tracking between capabilities, skills, models, knowledge, runtimes, workflows, and policies.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for dependency tracking between capabilities, skills, models, knowledge, runtimes, workflows, and policies.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityDependenciesDefined
- CapabilityDependenciesRegistered
- CapabilityDependenciesMatched
- CapabilityDependenciesInvoked
- CapabilityDependenciesRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Dependencies has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

Book 03.05.09 - Capability Lifecycle

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-009

Purpose

Define capability proposal, draft, review, approval, active use, deprecation, retirement, and replacement as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for capability proposal, draft, review, approval, active use, deprecation, retirement, and replacement. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for capability proposal, draft, review, approval, active use, deprecation, retirement, and replacement.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for capability proposal, draft, review, approval, active use, deprecation, retirement, and replacement.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityLifecycleDefined
- CapabilityLifecycleRegistered
- CapabilityLifecycleMatched
- CapabilityLifecycleInvoked
- CapabilityLifecycleRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Lifecycle has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

Book 03.05.10 - Capability DNA

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-010

Purpose

Define capability DNA that captures repeatable patterns, preferences, constraints, domain fit, and evolution metadata as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for capability DNA that captures repeatable patterns, preferences, constraints, domain fit, and evolution metadata. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for capability DNA that captures repeatable patterns, preferences, constraints, domain fit, and evolution metadata.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for capability DNA that captures repeatable patterns, preferences, constraints, domain fit, and evolution metadata.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityDnaDefined
- CapabilityDnaRegistered
- CapabilityDnaMatched
- CapabilityDnaInvoked
- CapabilityDnaRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability DNA has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-11-capability-benchmarks.md

Book 03.05.11 - Capability Benchmarks

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-011

Purpose

Define benchmarking capability quality, reliability, cost, latency, safety, and business impact as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for benchmarking capability quality, reliability, cost, latency, safety, and business impact. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for benchmarking capability quality, reliability, cost, latency, safety, and business impact.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for benchmarking capability quality, reliability, cost, latency, safety, and business impact.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityBenchmarksDefined
- CapabilityBenchmarksRegistered
- CapabilityBenchmarksMatched
- CapabilityBenchmarksInvoked
- CapabilityBenchmarksRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Benchmarks has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010

- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-12-capability-quality.md

Book 03.05.12 - Capability Quality

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-012

Purpose

Define quality controls for correctness, repeatability, explainability, reviewability, and degradation handling as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for quality controls for correctness, repeatability, explainability, reviewability, and degradation handling. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for quality controls for correctness, repeatability, explainability, reviewability, and degradation handling.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for quality controls for correctness, repeatability, explainability, reviewability, and degradation handling.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityQualityDefined
- CapabilityQualityRegistered
- CapabilityQualityMatched
- CapabilityQualityInvoked
- CapabilityQualityRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Quality has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-13-capability-governance.md

Book 03.05.13 - Capability Governance

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-013

Purpose

Define governance of capability ownership, policy checks, approvals, exceptions, and release authority as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for governance of capability ownership, policy checks, approvals, exceptions, and release authority. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for governance of capability ownership, policy checks, approvals, exceptions, and release authority.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for governance of capability ownership, policy checks, approvals, exceptions, and release authority.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityGovernanceDefined
- CapabilityGovernanceRegistered
- CapabilityGovernanceMatched
- CapabilityGovernanceInvoked
- CapabilityGovernanceRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Governance has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-14-capability-security.md

Book 03.05.14 - Capability Security

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-014

Purpose

Define security controls for capability invocation, protected actions, data access, secrets, tools, and tenant isolation as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for security controls for capability invocation, protected actions, data access, secrets, tools, and tenant isolation. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for security controls for capability invocation, protected actions, data access, secrets, tools, and tenant isolation.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for security controls for capability invocation, protected actions, data access, secrets, tools, and tenant isolation.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilitySecurityDefined
- CapabilitySecurityRegistered
- CapabilitySecurityMatched
- CapabilitySecurityInvoked
- CapabilitySecurityRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Security has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-15-capability-events.md

Book 03.05.15 - Capability Events

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-015

Purpose

Define canonical events for registration, matching, invocation, completion, failure, benchmark, governance, and retirement as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for canonical events for registration, matching, invocation, completion, failure, benchmark, governance, and retirement. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for canonical events for registration, matching, invocation, completion, failure, benchmark, governance, and retirement.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for canonical events for registration, matching, invocation, completion, failure, benchmark, governance, and retirement.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityEventsDefined
- CapabilityEventsRegistered
- CapabilityEventsMatched
- CapabilityEventsInvoked
- CapabilityEventsRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Events has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-16-capability-storage.md

Book 03.05.16 - Capability Storage

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-016

Purpose

Define logical storage for capability contracts, versions, runs, evidence, benchmarks, dependencies, and audit metadata as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for logical storage for capability contracts, versions, runs, evidence, benchmarks, dependencies, and audit metadata. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for logical storage for capability contracts, versions, runs, evidence, benchmarks, dependencies, and audit metadata.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for logical storage for capability contracts, versions, runs, evidence, benchmarks, dependencies, and audit metadata.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityStorageDefined
- CapabilityStorageRegistered
- CapabilityStorageMatched
- CapabilityStorageInvoked
- CapabilityStorageRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Storage has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-17-capability-metrics.md

Book 03.05.17 - Capability Metrics

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-017

Purpose

Define metrics for adoption, quality, performance, cost, reliability, governance, and business value as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for metrics for adoption, quality, performance, cost, reliability, governance, and business value. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for metrics for adoption, quality, performance, cost, reliability, governance, and business value.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for metrics for adoption, quality, performance, cost, reliability, governance, and business value.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityMetricsDefined
- CapabilityMetricsRegistered
- CapabilityMetricsMatched
- CapabilityMetricsInvoked
- CapabilityMetricsRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Metrics has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-18-capability-testing.md

Book 03.05.18 - Capability Testing

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-018

Purpose

Define testing strategy for capability contracts, matching, ranking, lifecycle, security, quality, and benchmarks as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for testing strategy for capability contracts, matching, ranking, lifecycle, security, quality, and benchmarks. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for testing strategy for capability contracts, matching, ranking, lifecycle, security, quality, and benchmarks.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for testing strategy for capability contracts, matching, ranking, lifecycle, security, quality, and benchmarks.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityTestingDefined
- CapabilityTestingRegistered
- CapabilityTestingMatched
- CapabilityTestingInvoked
- CapabilityTestingRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Testing has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-19-capability-acceptance.md

Book 03.05.19 - Capability Acceptance

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-019

Purpose

Define acceptance gates for Capability System blueprint completeness, governance readiness, testability, and release evidence as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for acceptance gates for Capability System blueprint completeness, governance readiness, testability, and release evidence. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for acceptance gates for Capability System blueprint completeness, governance readiness, testability, and release evidence.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for acceptance gates for Capability System blueprint completeness, governance readiness, testability, and release evidence.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.
- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityAcceptanceDefined
- CapabilityAcceptanceRegistered
- CapabilityAcceptanceMatched
- CapabilityAcceptanceInvoked
- CapabilityAcceptanceRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Acceptance has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.

book-03-05-20-capability-roadmap.md

Book 03.05.20 - Capability Roadmap

Version: v7 draft

Status: Draft

Document ID: MAL-V03-B03-05-020

Purpose

Define the implementation-facing roadmap for moving Capability System from blueprint to specifications and governed runtime behavior as part of the Mariam Capability System blueprint.

Scope

This document covers architecture-level responsibilities, contracts, data, events, controls, metrics, tests, and acceptance rules for the implementation-facing roadmap for moving Capability System from blueprint to specifications and governed runtime behavior. It does not implement system code or approve unreviewed automation.

Responsibilities

- Establish the canonical boundary for the implementation-facing roadmap for moving Capability System from blueprint to specifications and governed runtime behavior.
- Convert repeatable work patterns into governed, reusable capability contracts.
- Coordinate with Enterprise Core, AI Society, Knowledge System, and future Workflow System interfaces.
- Provide implementation-ready constraints for future capability specifications and runtime execution.

Inputs

- Enterprise Constitution enterprise, AI, quality, security, governance, and engineering principles.
- Master Roadmap Phase 05 Capability System requirements.
- Enterprise Core identity, runtime, event, storage, security, and observability contracts.
- AI Society skills, agents, planning, validation, and governance needs.
- Knowledge System retrieval, provenance, and quality signals.

Outputs

- Blueprint contract for the implementation-facing roadmap for moving Capability System from blueprint to specifications and governed runtime behavior.
- Capability records, dependency links, requirements, benchmarks, lifecycle states, and governance evidence.
- Traceable inputs for Volume 06 specifications and Volume 13 Capability System expansion.
- Acceptance evidence for matching, ranking, invocation, quality, security, and lifecycle gates.

Interfaces

- Enterprise Core object manager, runtime manager, event bus, storage, security, and observability interfaces.
- AI Society agent, skill, capability, execution, validation, and governance interfaces.
- Knowledge System search, graph, metadata, quality, and provenance interfaces.

- Future DNA Operating System, Workflow System, Marketplace, and Evolution System interfaces.

Events

- CapabilityRoadmapDefined
- CapabilityRoadmapRegistered
- CapabilityRoadmapMatched
- CapabilityRoadmapInvoked
- CapabilityRoadmapRetired

Storage

- Capability definitions with owner, version, state, input contract, output contract, constraints, and permission metadata.
- Capability graph records for prerequisites, dependencies, alternatives, compositions, and domain fit.
- Invocation, benchmark, review, exception, and acceptance evidence.
- Audit metadata linking capability use to user intent, agent role, workflow context, model choice, and policy status.

Security

- Apply least privilege to capability invocation, tool access, model use, knowledge retrieval, and protected actions.
- Deny capability execution when required policies, permissions, data classifications, or review gates are missing.
- Classify capability inputs, outputs, logs, benchmarks, and evidence before storage or sharing.
- Audit privileged capability registration, invocation, exception, deprecation, and replacement decisions.

Metrics

- Capability discovery rate, match precision, ranking quality, invocation success, and fallback rate.
- Quality score, benchmark pass rate, review rejection rate, defect rate, and rework count.
- Cost, latency, throughput, usage by domain, adoption by role, and business outcome contribution.
- Security denial count, governance exception count, stale capability count, and deprecation completion rate.

Tests

- Contract tests for capability definitions, requirements, dependencies, and event envelopes.
- Matching and ranking evaluation tests with known work-intent scenarios.
- Security tests for protected actions, tenant isolation, tool permissions, and data classification.
- Lifecycle tests for proposal, approval, active use, deprecation, retirement, and replacement.

Acceptance Criteria

- Capability Roadmap has explicit contracts, ownership, lifecycle, security, metrics, and governance controls.
- Events, storage records, metrics, and tests are defined well enough for downstream specifications.
- Capability use remains traceable to approved requirements, policies, evidence, and runtime context.
- Traceability links this blueprint to Volume 00, Volume 02, Book 03.01, Book 03.03, and Book 03.04.

Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-03-008
- MAL-V03-B03-04-015

Version History

Version	Date	Status	Notes
v7 draft	2026-06-29	Draft	Initial Capability System blueprint content for Mariam Architecture Library v7.